UNIVERSIDAD NACIONAL DE LOJA

CARRERA DE COMPUTACIÓN

Algoritmos, Análisis y Programación Paralela

Septiembre 2025 – Febrero 2026

Actividad en Contacto con el Docente # 5

Tema: Actividad en clase Patrones algorítmicos en paralelo y Análisis del rendimiento.

Integrantes:

- Luis Alberto Medina Chamba.
- Gerardo Israel Herrera Campoverde.
- Bayron Alejandro Jimenez Cango.
- Alexis Gradyn Ludeña Cueva.
- Nayely Cruzcaya Ramirez Herrera.

Fecha: 23 de enero 2026

Título: Análisis Paralelo de Imágenes Satelitales para Detección de Incendios Forestales.

1. **Contexto Real:**

Las agencias ambientales y de gestión de riesgos (por ejemplo, servicios forestales, protección civil y agencias espaciales) utilizan imágenes satelitales para detectar incendios forestales en tiempo casi real.

Cada imagen puede tener:

- Resolución: 10.000 × 10.000 píxeles
- Bandas espectrales múltiples (infrarrojo, visible)
- Miles de imágenes por día.

Problema:

El procesamiento secuencial es demasiado lento para emitir alertas tempranas.

2. **Problema Computacional:**

Para la solución del problema computacional, se requiere de un sistema que cumpla con las siguientes características :

- Lea gran volumen de datos computacionales.
- Analice pixel a pixel.
- Detecta las anomalías mediante la comparación de un umbral.
- Detecte incendios y los registre.
- Genera mapas de riesgo.
- Reduzca el tiempo en comparación al método secuencial.

3. **Modelo Secuencial:**

Código secuencial

```
#include <iostream>

#include <vector>
```

```
#include <cstdlib>
#include <ctime>
#include <chrono>

/*
  Análisis secuencial de imágenes satelitales
  Detección de incendios forestales
  Modelo: Secuencial
*/

#define UMBRAL 45.0

int main(int argc, char* argv[]) {

    if (argc < 3) {
        std::cout << "Uso: " << argv[0]
                  << " <filas> <columnas>\n";
        return 1;
    }

    int filas = atoi(argv[1]);
    int columnas = atoi(argv[2]);

    int total_pixeles = filas * columnas;

    std::vector<double> imagen(total_pixeles);
    std::vector<int> mapa_incendios(total_pixeles, 0);

    srand(time(NULL));

    // Inicialización de temperaturas
```

```
    for (int i = 0; i < total_pixeles; i++) {
        imagen[i] = (double)(rand() % 60);
    }

    int incendios_detectados = 0;

    // Inicio medición
    auto inicio = std::chrono::high_resolution_clock::now();

    // Procesamiento SECUENCIAL
    for (int i = 0; i < total_pixeles; i++) {
        if (imagen[i] > UMBRAL) {
            mapa_incendios[i] = 1;
            incendios_detectados++;
        }
    }

    // Fin medición
    auto fin = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> tiempo = fin - inicio;

    std::cout << "=== SECUENCIAL ===\n";
     std::cout << "Resolución: " << filas << " x " << columnas <<
"\n";
     std::cout << "Incendios detectados: " << incendios_detectados
<< "\n";
    std::cout << "Tiempo: " << tiempo.count() << " segundos\n";

    return 0;
}
```

```
root@DESKTOP-0HPOI33:~/Ramirez_Nayely_Fuente# g++ deteccion_incendios_secuencial.cpp -o secuencial
root@DESKTOP-0HPOI33:~/Ramirez_Nayely_Fuente# ./secuencial 10000 10000
=== SECUENCIAL ===
Resolución: 10000 x 10000
Incendios detectados: 23336054
Tiempo: 1.26088 segundos
root@DESKTOP-0HPOI33:~/Ramirez_Nayely_Fuente#
```

Fig 1 - Ejecución y resultado del Modelo Secuencial

4. **Análisis de Paralelismo Natural:**

MPI (Memoria Distribuida):

La detección de incendios a partir de píxeles de una imagen satelital, es un problema que puede resolverse desde partes completamente independientes, lo que lo hace naturalmente paralelo, a continuación se muestra las partes del código donde se asegura la paralelización de la solución.

- Independencia de datos, cada pixel se maneja de forma independiente

```
if (imagen_local[i] > UMBRAL) {
    incendios_local[i] = 1;
    incendios_proceso++;
}
```

Fig 2 - Manejo individual de cada pixel.

- La imagen total se divide entre los procesos

```
int pixeles_locales = total_pixeles / size;
```

Fig 3 - División de la imagen total en base a los procesos.

- Comunicación mínima, la única comunicación se da al final mediante

```
MPI_Reduce(&incendios_proceso, &incendios_totales,
           1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

double fin = MPI_Wtime();
```

Fig 4 - Uso de MPI_Reduce para la comunicación mínima entre el procesamiento.

Open MP (Memoria Compartida):

Al igual que el uso de la librería MPI, la aplicación del algoritmo sigue siendo naturalmente paralelo, la diferencia no está en el algoritmo, sino en cómo el modelo de paralelismo que se usa para implementarlo.

- Independencia de datos, cada pixel se maneja de forma independiente

-

```
if (imagen_local[i] > UMBRAL) {
    incendios_local[i] = 1;
    incendios_proceso++;
}
```

- **Fig 5** - Manejo individual de cada pixel.

- La imagen total se divide entre las filas y columnas.

```
int total_pixeles = filas * columnas;
```

Fig 6 - Imagen total en base a las filas y columnas totales.

¿Qué cambia con la implementación usando la librería MPI?

- Todos los hilos comparten la misma memoria
- No necesitan dividirse manualmente los datos
- No existe mensaje (MPI_Reduce), solo una reducción automática.

```
#pragma omp parallel for reduction(+:incendios_detectados)
for (int i = 0; i < total_pixeles; i++) {
    if (imagen[i] > UMBRAL) {
        mapa_incendios[i] = 1;
        incendios_detectados++;
    }
}
```

Fig 7 - Método de paralelización usando la librería OpenMP.

Pero cada iteración sigue siendo independiente y el paralelismo es natural por el diseño del problema y la solución aplicada.

5. **Diseño Paralelo (Patrones Aplicados):**

Patrón 1: Paralelismo de Datos.

El Paralelismo de Datos es un patrón clave que implica la distribución de grandes volúmenes de datos para su procesamiento simultáneo entre múltiples hilos o procesos. En este contexto, su aplicación se centra en dividir el total de píxeles de una imagen en segmentos iguales, asignando a cada porción a un hilo o proceso disponible. Esto se ilustra claramente en las implementaciones de código desarrolladas con OpenMP y MPI.

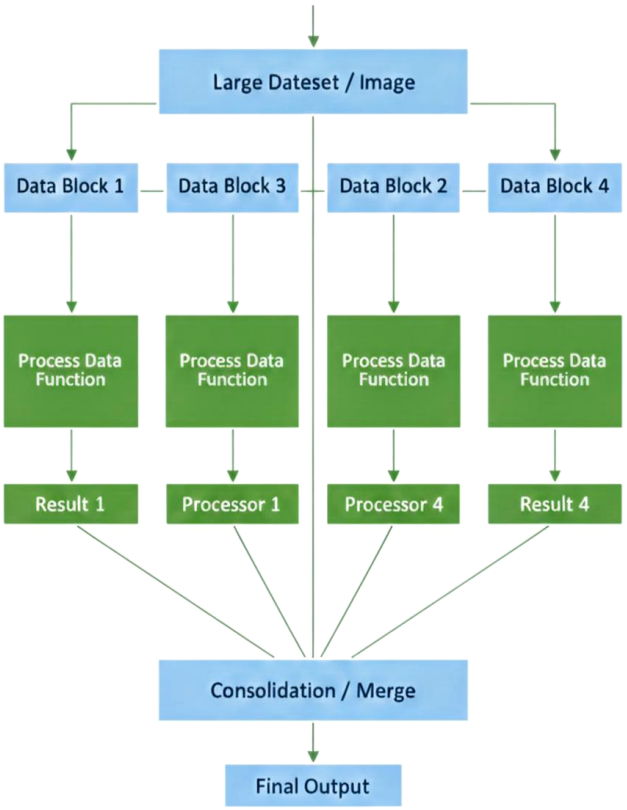


Fig. 8 - Flujo del patrón de paralelismo de datos.

Cada unidad de procesamiento recibe una fracción de los píxeles totales y ejecuta de forma paralela la misma operación: verificar si la temperatura de cada píxel supera el umbral de detección (45°C).

Este patrón garantiza que el trabajo se distribuya equitativamente entre los hilos o procesos, permitiendo que múltiples píxeles sean analizados simultáneamente y acelerando significativamente el tiempo de ejecución en comparación con el procesamiento secuencial.

Patrón 2: Pipeline.

El patrón Pipeline organiza el procesamiento en etapas encadenadas, donde cada etapa realiza una parte del trabajo y pasa el resultado a la siguiente.

Este flujo de trabajo describe un modelo de procesamiento en pipeline, diseñado para la detección eficiente de incendios forestales mediante computación distribuida. Cada etapa funciona de manera coordinada: el rango 0 actúa como el motor de datos generando la simulación térmica; el rango 1 limpia y prepara la señal mediante filtros; el rango 2 ejecuta la lógica crítica de detección comparando cada píxel contra un umbral de riesgo; y finalmente, el rango 3 consolida los hallazgos para presentar un reporte cuantitativo. Esta arquitectura permite que cada proceso se especialice en una tarea única, optimizando el flujo de información desde la captura del dato bruto hasta la obtención de resultados procesados en tiempo real.

Etapa 1: Generación de datos

- El proceso 0 produce los datos
- Luego los envía al siguiente proceso
- Inicio del pipeline

```
if (rank == 0) {

   imagen[i] = rand_r(&seed) % 60;

   MPI_Send(imagen.data(), pixeles, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);

}
```

Etapa 2: Preprocesamiento

- Recibe los datos de la etapa anterior
- Los procesa
- Los pasa a la siguiente etapa

```
if (rank == 1) {

        MPI_Recv(imagen.data(), pixeles, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

   imagen_filtrada[i] = imagen[i] * 1.05;

   MPI_Send(imagen_filtrada.data(), pixeles, MPI_DOUBLE, 2, 0, MPI_COMM_WORLD);

}
```

Etapa 3: Detección de incendios

- Analiza los datos
- Genera un resultado intermedio
- Continúa el flujo

```
if (rank == 2) {

        MPI_Recv(imagen_filtrada.data(), pixeles, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

   incendios[i] = (imagen_filtrada[i] > UMBRAL) ? 1 : 0;

   MPI_Send(incendios.data(), pixeles, MPI_INT, 3, 0, MPI_COMM_WORLD);

}
```

Etapa 4: Conteo Final

- Consume el resultado final
- Termina el pipeline

```
if (rank == 3) {

        MPI_Recv(incendios.data(), pixeles, MPI_INT, 2, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

   total += incendios[i];

}
```

Código Completo:

```
#include <mpi.h>

#include <iostream>

#include <vector>

#include <cstdlib>

#define UMBRAL 45.0

int main(int argc, char* argv[]) {

   MPI_Init(&argc, &argv);

   int rank, size;

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    if (size != 4) {
        if (rank == 0)
            std::cout << "Este programa requiere exactamente 4 procesos.\n";
        MPI_Finalize();
        return 1;
    }
    if (argc < 2) {
        if (rank == 0)
            std::cout << "Uso: mpirun -np 4 " << argv[0] << " <pixeles>\n";
        MPI_Finalize();
        return 1;
    }

    int pixeles = atoi(argv[1]);
    std::vector<double> imagen(pixeles);
    std::vector<double> imagen_filtrada(pixeles);
    std::vector<int> incendios(pixeles);
    double inicio = MPI_Wtime();

    // ETAPA 1: Generación de imagen
    if (rank == 0) {
        unsigned int seed = 123;
        for (int i = 0; i < pixeles; i++)
            imagen[i] = rand_r(&seed) % 60;
        MPI_Send(imagen.data(), pixeles, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
    }

    // ETAPA 2: Preprocesamiento
    if (rank == 1) {
                MPI_Recv(imagen.data(), pixeles, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        for (int i = 0; i < pixeles; i++)
            imagen_filtrada[i] = imagen[i] * 1.05; // simulación de filtro
        MPI_Send(imagen_filtrada.data(), pixeles, MPI_DOUBLE, 2, 0, MPI_COMM_WORLD);
    }
```

```
    // ETAPA 3: Detección de incendios
    if (rank == 2) {
        MPI_Recv(imagen_filtrada.data(), pixeles, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        for (int i = 0; i < pixeles; i++)
            incendios[i] = (imagen_filtrada[i] > UMBRAL) ? 1 : 0;
        MPI_Send(incendios.data(), pixeles, MPI_INT, 3, 0, MPI_COMM_WORLD);
    }

    // ETAPA 4: Conteo final
    if (rank == 3) {
        MPI_Recv(incendios.data(), pixeles, MPI_INT, 2, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        int total = 0;
        for (int i = 0; i < pixeles; i++)
            total += incendios[i];
        double fin = MPI_Wtime();
        std::cout << "=== MPI PIPELINE ===\n";
        std::cout << "Pixeles procesados: " << pixeles << "\n";
        std::cout << "Incendios detectados: " << total << "\n";
        std::cout << "Tiempo: " << fin - inicio << " segundos\n";
    }
    MPI_Finalize();
    return 0;
}
```

Ejecución del código:

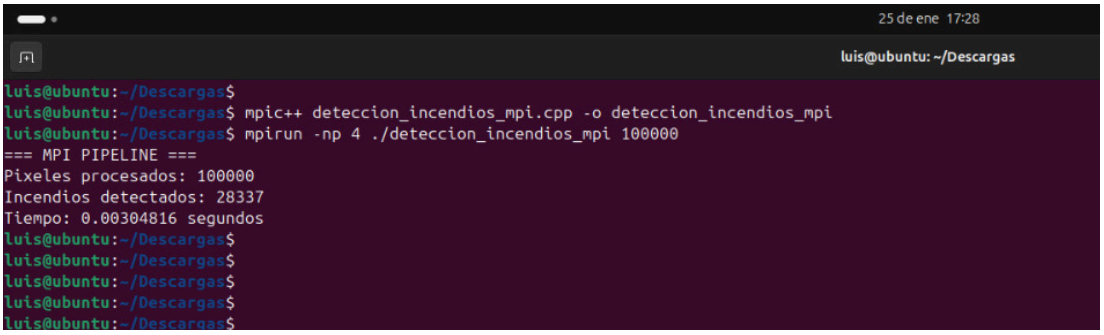


Fig 9 - Ejecución de MPI PIPELINE

Patrón 3: Map - Reduce.

- Map: cada proceso analiza una parte de los datos y produce un resultado parcial.
- Reduce: los resultados parciales se combinan en un resultado global.

Este flujo de trabajo describe un modelo de procesamiento basado en el patrón Map–Reduce, orientado a la detección eficiente de incendios forestales mediante computación distribuida. En la fase *Map*, cada proceso MPI analiza en paralelo una partición independiente de la imagen satelital, evaluando los valores térmicos de los píxeles y generando un conteo parcial de posibles incendios según un umbral predefinido. Posteriormente, en la fase *Reduce*, estos resultados parciales son combinados mediante una operación de reducción para obtener un conteo global de incendios detectados. Esta arquitectura permite escalar el procesamiento de grandes volúmenes de datos, aprovechando el paralelismo masivo y simplificando la agregación de resultados en un único reporte final.

Map: procesamiento en paralelo

```
for (int i = 0; i < pixeles_locales; i++) {

   imagen_local[i] = rand_r(&seed) % 60;

   if (imagen_local[i] > UMBRAL)

      incendios_locales++;

}
```

Reduce: combinación de resultados

```
MPI_Reduce(&incendios_locales, &incendios_totales,

         1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

Proceso raíz (resultado final)

```
if (rank == 0) {

   std::cout << "Incendios detectados: " << incendios_totales << "\n";

}
```

Código Completo:

```
#include <mpi.h>

#include <iostream>

#include <vector>

#include <cstdlib>

#define UMBRAL 45.0


int main(int argc, char* argv[]) {
```

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
if (argc < 3) {
    if (rank == 0)
        std::cout << "Uso: mpirun -np <procesos> "
                  << argv[0] << " <filas> <columnas>\n";
    MPI_Finalize();
    return 1;
}

int filas = atoi(argv[1]);
int columnas = atoi(argv[2]);
int total_pixeles = filas * columnas;
int pixeles_locales = total_pixeles / size;
std::vector<double> imagen_local(pixeles_locales);
unsigned int seed = rank;
// MAP: inicialización y detección local
int incendios_locales = 0;

for (int i = 0; i < pixeles_locales; i++) {
    imagen_local[i] = rand_r(&seed) % 60;
    if (imagen_local[i] > UMBRAL)
        incendios_locales++;
}
double inicio = MPI_Wtime();

// REDUCE: combinar resultados
int incendios_totales = 0;

MPI_Reduce(&incendios_locales, &incendios_totales,
           1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
double fin = MPI_Wtime();
if (rank == 0) {
```

```
        std::cout << "=== MPI MAP-REDUCE ===\n";
        std::cout << "Resolución: " << filas << " x " << columnas << "\n";
        std::cout << "Procesos: " << size << "\n";
        std::cout << "Incendios detectados: " << incendios_totales << "\n";
        std::cout << "Tiempo: " << fin - inicio << " segundos\n";
    }
    MPI_Finalize();
    return 0;
}
```

Ejecución de código:

```
25 de ene 17:32
luis@ubuntu: ~/Descargas
luis@ubuntu:~/Descargas$ mpic++ deteccion_incendios_mpi.cpp -o deteccion_incendios_mpi
luis@ubuntu:~/Descargas$ mpirun -np 4 ./deteccion_incendios_mpi 1000 1000
=== MPI MAP-REDUCE ===
Resolución: 1000 x 1000
Procesos: 4
Incendios detectados: 233185
Tiempo: 2.1636e-05 segundos
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
luis@ubuntu:~/Descargas$
```

Fig 10 - Ejecución de MPI MAP REDUCE

6. Implementación Técnica (en C++):

A continuación, se presenta los resultados de los códigos originales en C++ en el sistema operativo Windows 10.:

Open MP (Memoria Compartida):

```
C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>g++ deteccion_incendios_openmp.cpp -o deteccion_incendios_openmp -fopenmp -std=c++11

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 1
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 1
Incendios detectados: 23328013
Tiempo: 0.824 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 4
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 4
Incendios detectados: 23328013
Tiempo: 0.286 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 8
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 8
Incendios detectados: 23328013
Tiempo: 0.282 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 16
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 16
Incendios detectados: 23328013
Tiempo: 0.27 segundos
```

Fig 11 - Ejecución de OPENMP

En la figura , se observa los resultados obtenidos después de ingresar como parámetros una matriz de 10000 filas y 10000 columnas, y variando los hilos entre 1,4,8 y 16. Se aprecia una mejora

considerable en el tiempo de ejecución de 1 a 4 hilos, ya que el número de hilos coincide con la cantidad de procesadores lógicos disponibles en el sistema (4), permitiendo un aprovechamiento eficiente del paralelismo. Sin embargo, no existe mucha diferencia entre 8 y 16 ya que se supera la capacidad real del hardware, provocando sobrecarga por la gestión de hilos y cambios de contexto, lo que limita la escalabilidad del rendimiento.

MPI (Memoria Distribuida)MPI (Memoria Distribuida)

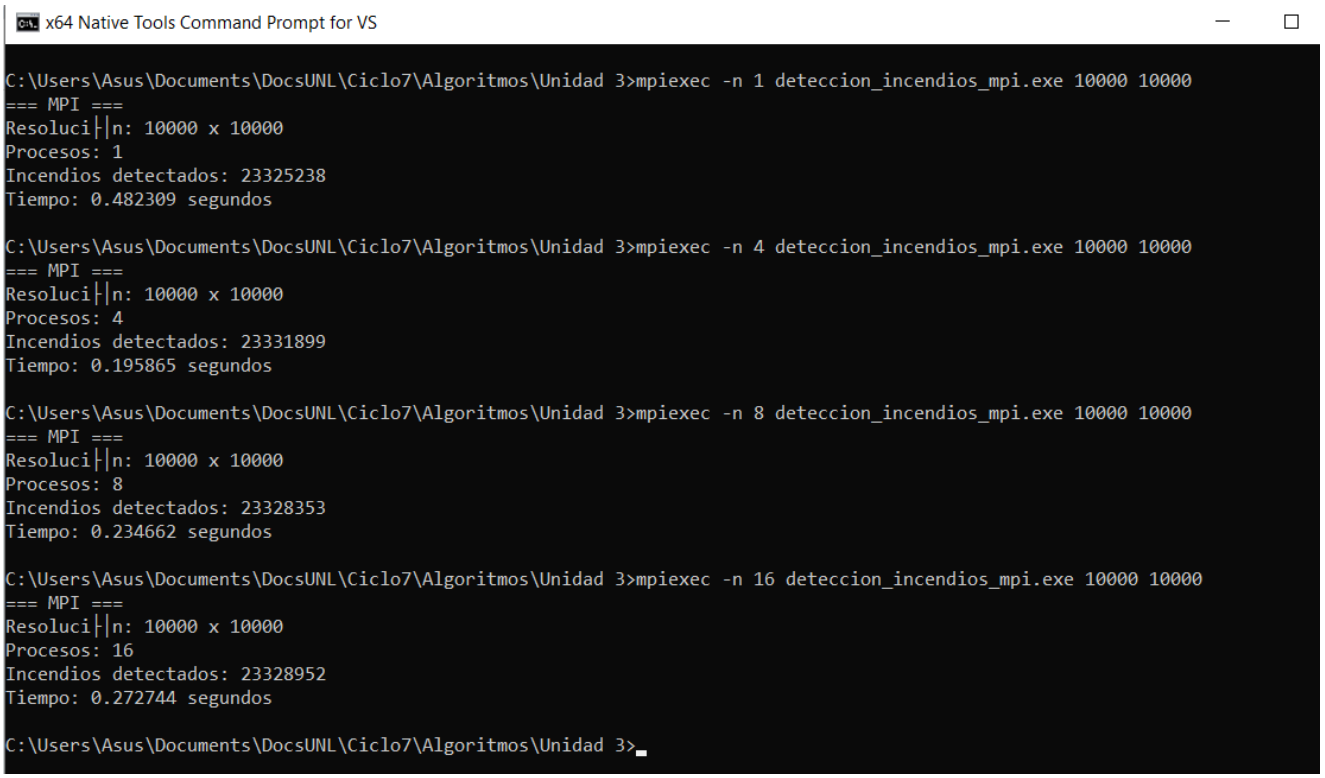


Fig 12 - Ejecución de MPI

Como se muestra en la figura , se aplicaron los mismos parámetros de la ejecución con OpenMP, pero, en este caso, con la lógica y características de MPI. De primera, se nota la mejora en la ejecución con un hilo (0.48 segundos) frente a OpenMP (0.82 segundos), sin embargo, nuevamente, no existe mejora entre 8 y 16 hilos debido al mayor número de hilos que requieren mensajes de envío y recepción, propio de MPI (Interfaz de paso de mensajes).

En el siguiente apartado se especifican los valores resultantes de la ejecución.

7. Análisis de Rendimiento:

En la tabla 1 y 2 se muestran los valores obtenidos en la ejecución de los códigos con una matriz de 10000 x 10000. Cabe recalcar que se consideró 95% como porcentaje paralelizable de los algoritmos ya que solo la porción de inicialización, reducción y lectura de filas, columnas e hilos es secuencial, el procesamiento de cada píxel es paralelo e independiente.

Open MP (Memoria Compartida):

Núcleos	Tiempo (s)	Speed-up	Eficiencia
1	0.824	1.00	1.00
4	0.286	3.48	0.87
8	0.282	5.93	0.74
16	0.27	9.14	0.57

Tabla 1. Resultados de ejecución del código implementado con OpenMP.

MPI (Memoria Distribuida):

Núcleos	Tiempo (s)	Speed-up	Eficiencia
1	0.482	1.00	1.00
4	0.195	3.48	0.87
8	0.234	5.93	0.74
16	0.272	9.14	0.57

Tabla 2. Resultados de ejecución del código implementado con MPI.

Como se trabaja con el mismo número de núcleos y porción paralelizable, los valores de seed-up y eficiencia son los mismos en ambas tablas. Adicionalmente, cabe resaltar que aunque el speed-up sigue aumentando con más hilos, la eficiencia disminuye porque la fracción secuencial y la sobrecarga de paralelización limitan el aprovechamiento de recursos, especialmente en un equipo con solo 4 hilos lógicos.

8. **Aplicación de la Ley de Amdahl:**

La Ley de Amdahl define la máxima mejora de velocidad que puede lograrse mediante la paralelización de un sistema.

Fórmula:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

- **S:** Aceleración teórica (*Speed-up*).
- **P:** Fracción del código que puede ser paralelizada.
- **1 - P:** Fracción secuencial.
- **N:** Número de núcleos o procesadores utilizados.

Con base en el porcentaje paralelizable del 95% y los hilos o procesos utilizados en la Tabla 3, los cálculos teóricos son:

Núcleos	Speed-up OpenMP	Speed-up MPI
1	1.00	1.00
4	3.48	3.48
8	5.93	5.93
16	9.14	9.14

Tabla 3. Resultados de la aplicación de la Ley de Amdahl.

En la tabla 3, se observa una mejora significativa en el *Speed-up* al pasar de 1 a 4 núcleos se obtiene una mejora considerable (3.48), pero a medida que se incrementan los núcleos a 8 y 16, el speed-up crece a un ritmo cada vez menor (5.93 y 9.14 respectivamente), reflejando el límite teórico impuesto por la parte no paralelizable. El hecho de que OpenMP y MPI presenten valores idénticos sugiere que el factor dominante no es el modelo de paralelización, sino el porcentaje secuencial del algoritmo, confirmando que, según la Ley de Amdahl, agregar más núcleos produce beneficios decrecientes cuando la fracción secuencial permanece constante.

9. Discusión Académica: (Conclusión)

Los resultados obtenidos demuestran que la solución propuesta presenta una excelente escalabilidad, ya que el tiempo de ejecución disminuye notablemente al pasar del modelo secuencial a los modelos paralelos con OpenMP y MPI. El mayor beneficio se observa al incrementar el número de núcleos hasta el límite del hardware disponible, lo que evidencia que el paralelismo de datos permite aprovechar de manera efectiva los recursos del sistema.

El algoritmo requiere mínima sincronización, debido a que cada píxel de la imagen se procesa de forma independiente. Esto reduce la necesidad de coordinación entre hilos o procesos durante la etapa principal de cómputo, limitando la sincronización únicamente a la fase final de reducción de resultados. Como consecuencia, se disminuye la sobrecarga asociada al paralelismo y se mejora el rendimiento general.

Además, el uso de accesos a memoria locales contribuye a una ejecución más eficiente, ya que cada hilo o proceso trabaja sobre su propia porción de datos. Esto evita conflictos de acceso a memoria y mejora la eficiencia tanto en el modelo de memoria compartida como en el de memoria distribuida.

Finalmente, la solución se apoya en un patrón claro y reusable, principalmente el paralelismo de datos, el cual puede aplicarse fácilmente a otros problemas que requieran el procesamiento de grandes volúmenes de información de forma independiente. Esto demuestra que el diseño del algoritmo no solo es eficiente para la detección de incendios forestales, sino que también es adaptable a otros escenarios de computación paralela.

10. Preguntas para Analizar:

1. ¿Qué patrón paralelo domina el problema?

El patrón paralelo dominante del problema es el paralelismo de datos, ya que el análisis de detección de incendios se realiza sobre múltiples datos independientes aplicando la misma operación, lo que permite su ejecución simultánea tanto en MPI como en OpenMP.

2. ¿Dónde se pierde eficiencia al aumentar núcleos?

La pérdida de eficiencia al aumentar el número de núcleos ocurre principalmente debido a la sobrecarga de comunicación y sincronización, así como a una granularidad de trabajo cada vez más fina. En la implementación con MPI, el costo de operaciones colectivas como MPI_Reduce comienza a dominar el tiempo total de ejecución. En OpenMP, la contención de memoria limita la escalabilidad. Por lo tanto, a partir de cierto número de núcleos, el incremento de paralelismo no se traduce en una mejora proporcional del rendimiento.

3. ¿Por qué Amdahl limita el speed-up?

La Ley de Amdahl limita el *Speed-up* porque siempre existe una fracción del programa que debe ejecutarse de forma secuencial y no puede ser paralelizada; aunque se aumente el número de procesadores y la parte paralela se ejecute cada vez más rápido, el tiempo de ejecución de la parte secuencial permanece constante y termina dominando el tiempo total, imponiendo así un límite máximo al *Speed-up* alcanzable independientemente de los recursos disponibles.

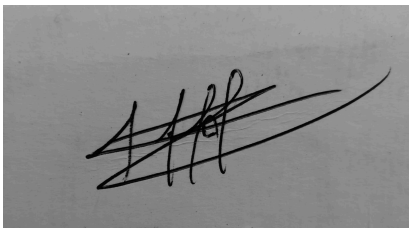
4. ¿Qué pasaría si la imagen fuera más grande?

Si la imagen satelital tuviera una mayor resolución, el número total de píxeles aumentaría, incrementando la carga de cómputo. Sin embargo, debido a que el programa aplica paralelismo de datos, la imagen se divide entre los procesos disponibles y cada uno analiza únicamente su porción local. Esto permite que el tiempo de procesamiento crezca de forma controlada, ya que el trabajo se distribuye entre los procesos. No obstante, si el tamaño de la imagen aumenta sin incrementar el número de procesos, cada proceso deberá manejar más datos, lo que puede aumentar el tiempo de ejecución y el consumo de memoria. Para mantener un buen rendimiento, sería conveniente escalar también el número de procesos.

5. ¿OpenMP o MPI? ¿Por qué?

La elección puede variar según el entorno en el que se implemente, pero en el caso de esta actividad, donde se ejecutaron los programas de forma independiente en los ordenadores de los miembros del equipo, la mejor opción sería OpenMP, ya que utiliza memoria compartida y puede ejecutarse en un solo equipo utilizando los hilos lógicos que tenga, aprovechando mejor los recursos y sin costo adicional de comunicación. Por otro lado, MPI introduce sobrecarga por envío y recepción de mensajes, lo que reduce su eficiencia cuando se excede el número de núcleos físicos disponibles.

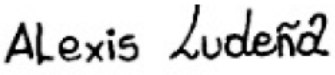
Por ello, OpenMP resulta más simple, eficiente y adecuado para este escenario.



Gerardo Israel Herrera Campoverde

Bayron Alejandro Jimenez Cango

Nayely Cruzcaya Ramirez Herrera



Alexis Gradyn Ludeña Cueva

Luis Alberto Medina Chamba